CS3103

Assignment 4 MiniProject

LiveRecall

"Never lose the lecture because you missed the last sentence."

Project	LiveRecall - Never lose the lecture because you missed the last sentence
Team size	4 students
Duration	3 weeks
Deadline	04-Nov-2026
Evaluation environment	Ubuntu Linux; public tests + instructor-only hidden performance evaluation
Final Demo & Technical Interview	Friday, 6 November 2026, 10:00 AM-6:00 PM; 15-minute team slots

Best projects may be considered for continuation as an FYP, research prototype, or open-source project.

1. Background and Objectives

Problem. During a live lecture, a student may miss a sentence because they are taking notes, momentarily distracted, unfamiliar with an accent, or unable to hear clearly. LiveRecall is a classroom companion that continuously displays live captions and keeps a short, recoverable history so that a student can immediately ask, "What did I miss?"

This is a networking and distributed-application assignment; no speech-recognition implementation is required for the mandatory work. The networking problem is to distribute a continuously changing, revision-based caption stream to many clients while maintaining correctness, freshness, bounded resource usage, and efficient recovery under slow clients, temporary disconnections, bursty updates, and constrained networks.

By completing this assignment, you should be able to:

- design and document a small application-layer protocol over WebSockets;
- manage revision-based state, deduplication, bounded history, and reconnect/resume semantics;
- apply backpressure and semantics-aware overload handling to a real-time application;
- measure P50/P95/P99 latency, caption freshness, completeness, and recovery behavior;
- use Linux traffic-control tools to create reproducible network impairment experiments;
- evaluate both system performance and application usability; and
- critically evaluate code produced with AI/coding assistants and defend your design in a viva.

2. System Overview

caption_source.py → LiveRecall Server → many student browser clients

The system has three logical parts:

- Caption source: the supplied deterministic caption_source.py generates reproducible caption segments and revisions for the mandatory application demonstration, public tests, and instructor performance evaluation.
- LiveRecall server: your implementation. It receives caption updates, maintains a bounded rolling history of recent finalized captions together with the latest state of active segments, and distributes updates to concurrent clients.
- Student client: a browser-based user interface that displays live captions, keeps recent context, supports "What did I miss?", and automatically reconnects.

Separation of concerns: The caption source, LiveRecall server, and browser client are separate components. The same submitted server and browser client must work with the deterministic caption_source.py used in the public tests and final evaluation.

3. Mandatory Application Functionality

#	Requirement	Minimum behavior
1	**Live captions**	Display incoming captions continuously. A provisional caption is an update with *final=false* and may later be replaced by a newer revision of the same segment. A final caption has *final=true* and represents the completed text for that segment. Provisional and final captions must be visually distinguishable. Example: Revision 1 (provisional): TCP uses Revision 2 (provisional): TCP uses cumulative Revision 3 (final): TCP uses cumulative acknowledgements.
2	**Revision replacement**	A newer revision of the same segment replaces the older revision. It must not appear as a duplicate line.
3	**What did I miss?**	The client must maintain a rolling local history of at least the previous 60 seconds of finalized captions. Selecting "What did I miss?" must initially display the previous 30 seconds, with options to view 15 s, 30 s, or 60 s. Under normal operation this must use captions already stored locally at the client; a server request is only needed to recover information the client genuinely missed, such as after a disconnection or late join.
4	**Automatic reconnect**	If the connection is lost, the client reconnects automatically and reports its connection state to the user.
5	**Incremental recovery**	After a short disconnection, the client recovers recent missed finalized captions without replaying the entire lecture.
6	**Late join**	A client joining an ongoing lecture can request recent context and then continue with live updates.
7	**No duplicate finals**	A finalized caption must not be shown twice after reconnect/recovery.
8	**Bounded history**	The server must maintain a bounded recent-history window sufficient to satisfy recovery/late-join requests of up to 60 seconds of finalized captions. It may keep a somewhat larger bounded window, but it must not keep an unbounded lecture transcript or unbounded per-client backlog solely to achieve eventual delivery.
9	**Concurrent clients**	The server must support many simultaneously connected clients. The supplied public tests use up to 100 clients; final evaluation may use substantially more.
10	**Slow-client isolation**	A slow receiver must not create unbounded delay or memory growth for normal clients.
11	**Graceful overload**	When a client cannot keep up, obsolete provisional revisions may be coalesced or discarded. The final caption for every completed segment must be preserved and delivered correctly. For a segment that is not yet final, the newest relevant revision should be preserved.

4. Required Protocol and Compatibility Interface

For automated testing, all submissions must implement the following compatibility interface exactly. You may extend the protocol, but the required fields and message meanings must remain compatible.

4.1 Source connection

The caption source connects to the server by WebSocket and first sends the following message. Mandatory tests use one active source and one session (session="L1") at a time; multi-session support, authentication, and TLS/WSS are not required.

```
{
 "type": "SOURCE_HELLO",
 "session": "L1"
}
```

Caption updates use:

```
{
 "type": "CAPTION_UPDATE",
 "session": "L1",
 "segment": 42,
 "revision": 3,
 "source_ts": 1760000000.123,
 "final": true,
 "text": "TCP uses cumulative acknowledgements."
}
```

- `segment` is a positive integer identifying successive caption segments. The instructor source uses monotonically increasing segment numbers starting from 1. The caption source in the starter kit completes a segment before beginning the next segment.
- `revision` is a positive integer that increases for newer versions of the same segment. Do not assume a fixed number of revisions per segment.
- source_ts is the source event timestamp in seconds since the Unix epoch.
- final=true means that the source declares the segment complete. The final caption is durable state: older revisions must never overwrite it. The instructor test source will not intentionally send a newer revision after a final caption for that segment.
- text contains the caption text.

At the end of a deterministic test, the source may send a SOURCE_DONE message. It is a test-control message; the server may ignore it after processing earlier updates and need not forward it to clients. Source messages may also contain extra JSON fields that are not part of the minimum interface (for example, a test-phase label). Your server must ignore unrecognized fields and continue operating normally.

4.2 Client connection and resume

A student/load-test client connects by WebSocket and first sends:

```
{
 "type": "CLIENT_HELLO",
 "client_id": 17,
 "session": "L1",
 "last_segment": 38,
 "last_revision": 3,
 "history_seconds": 30
}
```

For the required resume behavior, last_segment/last_revision identify the latest finalized segment state already held by the client, and history_seconds requests the amount of recent recovery context (up to 60 seconds). On `CLIENT_HELLO`, the server returns a `HISTORY` message, which may contain zero or more finalized captions. Items

must be ordered from oldest to newest and must not include finalized captions already reported by the client through `last_segment/last_revision`:

```
{
 "type": "HISTORY",
 "items": [ ... CAPTION_UPDATE objects ... ]
}
```

During live operation, the server sends CAPTION_UPDATE objects to clients. A client must never display an older revision after a newer revision of the same segment has already been accepted. A client with no previous state uses `last_segment=0` and `last_revision=0`.

4.3 Overload semantics

Core networking challenge: Under normal conditions, clients should receive the full sequence of caption revisions. Under overload, the server may suppress obsolete provisional revisions. However, it must preserve the final caption for every completed segment and the latest available revision of any currently active segment. A design that simply queues every revision indefinitely will perform poorly in the hidden slow-client and constrained-bandwidth tests.

5. Required Repository Structure and Server Entry Point

The performance evaluator will run every group using the same commands. Therefore, naming and command-line compatibility are mandatory.

```
submission_root/
├── solution_server.py       # REQUIRED evaluation entry point
├── requirements.txt         # REQUIRED Python dependencies
├── client/                  # browser client files
├── README.md                # setup and usage
├── AI_USAGE.md              # required AI/tool declaration
├── CONTRIBUTIONS.md         # required contribution statement
└── report.pdf               # technical report (4-5 pages)
```

The evaluator will start your server with:

```
python solution_server.py --port 8765
```

- *solution_server.py* must accept --port <integer>.
- The server must accept connections on the specified port. For local evaluation, clients will connect to ws://127.0.0.1:<port>. Binding to 0.0.0.0 is recommended (but not required) because it also makes the server reachable through other network interfaces of the Ubuntu VM.
- The server must start without interactive input and be ready within 10 seconds on the evaluation VM.
- All Python dependencies required by the *solution_server.py* must be listed in *requirements.txt*.
- The *solution_server.py* must not depend on cloud APIs, external Internet services, or an external database.
- Optional bonus-feature dependencies must not be required to install or start the mandatory solution_server.py evaluation path.
- Do not modify instructor test programs or assume knowledge of hidden timing/parameters.
- Additional files and implementation languages are allowed only if README.md gives a non-interactive setup that works on the specified Ubuntu environment. The required Python entry point must still work.

6. Mandatory Application Demonstration (Deterministic Caption Source)

The final demonstration must show the submitted LiveRecall application using the supplied deterministic caption_source.py. ASR, live speech recognition, and real-time transcription are not required for the mandatory assignment.

- Use the same solution_server.py and browser client that you submit for performance evaluation; the demonstration must not be a separate mock-up.
- Demonstrate at least two browser clients receiving a running caption stream from caption_source.py.
- Show provisional-to-final revision replacement, including that newer revisions replace older revisions rather than appearing as duplicate lines.
- Show "What did I miss?" with the default 30-second view and the selectable 15/30/60-second recall windows.
- Show a temporary client disconnect followed by automatic reconnect and incremental recovery with no duplicate final captions.
- Show a late-joining client obtaining recent context and then continuing with live updates.

Demonstration versus performance: The mandatory demo is assessed for correct and usable application behavior. Quantitative networking performance is evaluated separately with the deterministic public/hidden test harness. Do not infer benchmark performance from the visual demo alone.

7. Provided Starter/Test Materials

A small starter/test package will be provided. It is intentionally a test harness, not a complete solution. You are expected to build the server logic and application behavior.

File / folder	Purpose
START_HERE.md	Student-facing setup and public-test instructions.
solution_server.py	REQUIRED startup/CLI/WebSocket skeleton. Working naive WebSocket broadcaster providing the required connection and broadcast path. It deliberately omits the graded revision-state, recovery, bounded-buffering, slow-client isolation, and semantic-backpressure mechanisms. Teams implement the graded server logic while preserving the required command-line interface.
caption_source.py	Deterministic caption/revision generator used for reproducible public development tests.
load_clients.py	Scalable simulated WebSocket clients, including slow-reader and temporary-disconnect options.
run_demo.py	One-command public test runner. It starts the team solution_server.py, simulated clients, deterministic caption source, and result analysis.
analyse_results.py	Common metrics: P50/P95/P99 Caption Age, Fresh Caption Ratio, Fresh Final Caption Ratio, final-caption completeness, and Normal Update Coverage.
network_profiles/	Example Linux tc/netem scripts for delay/loss/bandwidth experiments. These are public examples, not the hidden final workload.
client/	Minimal browser connection/UI skeleton. Revision semantics, recall/history, reconnect/resume, deduplication, and usable rendering remain graded TODOs.
UBUNTU_SETUP.md	Ubuntu setup plus an optional Multipass workflow for macOS/Windows hosts.

File / folder	Purpose
submission_check.py	Checks required file names and verifies that python solution_server.py --port <port> starts and listens.
README_TEMPLATE.md / AI_USAGE_TEMPLATE.md / CONTRIBUTIONS_TEMPLATE.md	Templates for required submission documentation.
sample_transcript.txt	Small deterministic caption text source.
NETWORKING_PRIMER.md	Short introduction to the WebSocket/async networking concepts used in the assignment, including per-client queues, backpressure, bounded buffering, and revision coalescing.
PROTOCOL_EXAMPLES.md	Example message exchanges for normal connection, late join, disconnect/reconnect, and history recovery.
PUBLIC_TESTS.md	Explains the purpose of each public test, the expected observations, and how to interpret the reported metrics.
monitor_server.py	Optional helper for monitoring the server process, including memory usage during longer or stressed experiments.
bonus_asr/	Optional material for the real-lecture ASR bonus feature. It is not required for the mandatory assignment or performance evaluation.

8. Mandatory Public Tests and Measurements

At minimum, every team must run and report the following public tests using the supplied test harness without modifying its measurement logic. Run each test at least three times and report the median result. Public tests are for development; they do not reveal the final hidden workload.

Test	Required purpose
T1 - Basic correctness	20 clients; normal local network; verify revision/final delivery, no stale/duplicate state, and basic protocol correctness. The browser recall UI is demonstrated separately in Section 6.
T2 - Public scale	100 clients; normal local network; measure P50/P95/P99 Caption Age and completeness.
T3 - Slow clients	100 clients with 10% slow readers; demonstrate that normal clients remain responsive and resource usage remains bounded.
T4 - Reconnect	100 clients with 20% temporarily disconnected; verify incremental recovery and no duplicate final captions.
T5 - Network impairment	Use Ubuntu tc/netem to add delay/jitter/loss and explain the effect on WebSocket/TCP latency and tail behavior.

Recommended commands include:

```
python run_demo.py --clients 20 --duration 20 --runs 3
python run_demo.py --clients 100 --duration 40 --runs 3
python run_demo.py --clients 100 --duration 60 --runs 3 --slow-percent 10 --slow-ms 500
python run_demo.py --clients 100 --duration 60 --runs 3 --disconnect-percent 20 --disconnect-at 15 --disconnect-seconds 5
```

Required before/after comparison. For at least one public stress test, compare your final design with an earlier/basic configuration or with a major optimization disabled. Quantify the improvement and explain the trade-off. This is intended to show engineering evidence rather than only final benchmark numbers.

9. Performance Metrics

You must understand and report the following metrics. The final evaluator uses related metrics, but exact workload parameters are hidden.

Metric	Meaning
Caption Age	For automated performance tests: client receive time - source_ts for a caption event. Browser rendering/UI delay is not included in the hidden networking benchmark.
P50 Caption Age	median Caption Age: 50% of measured captions are at or below this value.
P95 Caption Age	tail-latency metric: 95% of measured captions are at or below this value.
P99 Caption Age	extreme tail-latency metric: 99% of measured captions are at or below this value.
Final Caption Completeness	unique final captions correctly received / unique final captions generated.
Fresh Caption Ratio (T)	Fraction of caption updates received by the automated client within threshold T. The default public threshold is 500 ms.
Normal Update Coverage	during unconstrained operation, fraction of source revisions received. This prevents a system from obtaining a high freshness score simply by discarding all provisional updates.
Recovery time	time from successful reconnect until the client has received the required recent recovery state.
Slow-client isolation	performance of normal clients while another population is deliberately slow.
Fresh Final Caption Ratio (T)	fraction of generated final captions that are correctly received within threshold T. This rewards fresh delivery of durable caption state even when obsolete provisional revisions are suppressed under overload.

10. Instructor-Only Hidden Performance Evaluation

The instructor will run each submission on a common Ubuntu evaluation environment.

The hidden evaluator may vary and combine:

- client population size (larger than the public test);
- network delay, jitter, and packet loss;
- slow-reader percentage and slow-reader rate;
- temporary disconnects and simultaneous reconnects;
- caption revision rate and burstiness; and
- available bandwidth.

All groups will be evaluated using the same hidden workload(s) and machine configuration. The instructor may repeat a workload and use the median result to reduce run-to-run noise.

Correctness gate: Performance optimizations do not compensate for incorrect state. If Final Caption Completeness is below 99%, the hidden-performance section is capped at 15/30. If it is below 95%, the hidden-performance section is capped at 5/30. A submission that cannot be started by the required command receives 0 for hidden performance until the start-up problem is resolved, subject to course late/assessment rules.

The two most difficult hidden challenges are intentionally designed so that a naive “queue every message forever” solution performs poorly:

1. Slow-reader semantic backpressure. Some clients consume more slowly than caption revisions are generated. High performance requires bounded queues and semantics-aware handling of superseded provisional revisions while keeping normal clients fresh and preserving all final captions.
2. Constrained-bandwidth semantic delivery. Available bandwidth may be lower than the bandwidth required to forward every provisional revision to every client. High performance requires graceful degradation: prioritize final captions and the latest useful state instead of building an increasingly stale backlog.

A third requirement prevents trivial "drop everything except finals" solutions: under unconstrained/normal conditions, the evaluator also checks revision/update coverage. Your overload policy should activate because of actual backpressure/lag and should return to full-update behavior when the client/network catches up.

Rapid convergence after overload. When a temporary bottleneck, slow-reader condition, or disconnection ends, a client should converge quickly to the newest useful state instead of spending a long time draining stale queued revisions. Hidden evaluation may include a recovery window after an impairment is removed.

Boundedness is part of correctness. A design that achieves eventual delivery by allowing unbounded per-client queues, history, or memory growth may be penalized even if it eventually delivers all messages.

11. Required Ubuntu Environment

Ubuntu Linux is compulsory for the reproducible networking/performance portion because the supplied impairment scripts and final evaluator use Linux traffic control (tc/netem). Most students should already have an Ubuntu environment from the earlier Mininet laboratory; that environment may be reused if it can run the supplied scripts.

Minimum software:

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip iproute2
```

- A native Ubuntu installation or the Ubuntu VM used for the Mininet lab is acceptable.
- If you do not already have a suitable Ubuntu VM, Canonical Multipass is a convenient headless VM manager available for macOS and Windows; it can create an Ubuntu VM with a small command-line workflow. Students may also use another full Ubuntu VM solution.
- WSL is not the reference grading environment; use a full Ubuntu VM if tc/netem behavior differs from the supplied tests.
- For the performance experiment, run the server and simulated clients inside the same Ubuntu VM; tc/netem can then be applied to the loopback interface (lo).

Example Multipass workflow (resource sizes are suggestions, not grading requirements):

```
multipass launch --name liverecall --cpus 2 --memory 4G --disk 15G
multipass shell liverecall
# inside Ubuntu:
sudo apt update
sudo apt install -y python3 python3-venv python3-pip iproute2

# when finished on the host:
multipass stop liverecall
```

Official Multipass installation guide: https://canonical.com/multipass/install

12. Submission Requirements

Submit one group zip_package/git_repository containing all of the following. Missing required files may prevent automated evaluation. ***Submit the package or repo URL to 'miniProject-Assignment4-submission' in Canvas.***

Deliverable	Requirement
Source repository / ZIP	All source code in the required repository structure; no generated virtual environment or model weights unless specifically approved.
README.md	Exact Ubuntu setup, dependency installation, how to start solution_server.py, browser client, and the public

Deliverable	Requirement
	deterministic tests.
report.pdf	4-5 pages excluding references/appendices. See required contents below.
Demo video	5-7 minutes showing the deterministic caption_source.py-driven application, including live revision replacement, "What did I miss?", and disconnect/reconnect recovery.
AI_USAGE.md	Tools used; what they were used for; one useful AI suggestion; and at least one generated suggestion/code fragment that was incorrect, inefficient, or required modification, with your explanation.
CONTRIBUTIONS.md	Named responsibilities, components/experiments/report sections, and agreed percentage contribution for each member.
Git history	Use the group repository throughout the project. Commit count alone is not graded; it is supporting evidence for the contribution statement/viva.

The report must include: architecture and protocol; revision/recovery semantics; backpressure/overload policy; public experiments; at least one before/after optimization comparison; P50/P95/P99/freshness/final-completeness results; discussion of failures and trade-offs; limitations; and a short future-work/FYP direction.

13. Grading Scheme (100 marks)

Component	Marks	What is assessed	Top-mark expectation
A. Usable application and deterministic demo	15	Browser client, live revision display, 15/30/60-second recall, reconnect/recovery, late join, and coherent usable workflow using caption_source.py.	Application behaves like a usable classroom tool, not merely a technically connected WebSocket demo.
B. Protocol correctness and robustness	15	Revision semantics, deduplication, bounded history/queues, reconnect recovery, slow-client isolation.	No stale/duplicate finals; bounded state and explicit overload policy.
C. Public experiments and analysis	10	Reproducible public tests, appropriate metrics, repeated runs, explanation of observed networking behavior.	Evidence-based explanation, not only screenshots/output.
D. Hidden performance evaluation	30	Correctness gate, normal update coverage, delay/loss freshness, slow-reader challenge, constrained-bandwidth challenge, reconnect/catch-up recovery.	Near-perfect final correctness plus strong freshness and rapid convergence under both advanced challenges.
E. Code/repository/report/reproducibility	10	Clean repository, README, dependencies, report quality, AI_USAGE.md and contributions.	Instructor can reproduce without guesswork.
F. Individual technical viva	20	Ability to explain design, code paths, logs/traces, metrics, failure modes and make/describe small changes. Individual adjustment may be applied.	Member demonstrates ownership and understanding beyond generated code.

13.1 Hidden-performance mark breakdown

Hidden component	Marks	Assessment

Hidden component	Marks	Assessment
Correctness and invariants	8	Final completeness, no stale/duplicate final state, server remains operational; bounded-state violations may reduce this component.
Normal update coverage and normal freshness	4	Full-update behavior when the system is not overloaded; prevents trivial final-only solutions.
Delay/loss tail freshness	2	P95/freshness under impaired TCP/WebSocket conditions and correct explanation of tail behavior.
Advanced Challenge 1: slow-reader semantic backpressure	6	Fresh useful state for slow clients while normal clients remain unaffected; bounded per-client queues and deliberate handling of obsolete revisions.
Advanced Challenge 2: constrained-bandwidth semantic delivery	6	Graceful degradation and fresh finals/latest state when forwarding every provisional revision promptly is impossible.
Reconnect and catch-up recovery	4	Correct incremental reconnect plus rapid convergence to current state after temporary outage/overload without significantly degrading unaffected clients.

Grade differentiation: Passing all public tests does not imply full hidden-performance marks. A complete conventional implementation can earn a good mark; top performance marks are intentionally difficult and require measurable semantic-backpressure/capacity handling, rapid catch-up, bounded state, and strong individual viva performance. Full marks in either 6-mark advanced hidden challenge are reserved for behavior that is materially stronger than a conventional queue-based implementation on the same hidden workload while preserving correctness. Optional bonus features do not compensate for weak networking design.

14. Use of Generative AI and Coding Assistants

Use of generative AI/coding assistants is permitted. You remain fully responsible for every submitted line of code, system behavior, experimental result, and claim.

- You must be able to explain and modify your implementation during the viva.
- AI-generated code that merely reproduces the baseline functionality does not by itself earn high marks.
- Hidden evaluation rewards engineering decisions that are validated under conditions not disclosed in advance.
- AI_USAGE.md is compulsory and must include critical evaluation of at least one AI-generated suggestion that you rejected or changed.

15. Final Demo and Technical Interview (Viva-Voce)

Date	Friday, 6 November 2026
Published window	10:00 AM-6:00 PM
Slot length	15 minutes per team

Booking link	https://nusu-my.sharepoint.com/:x:/g/personal/dcsab_nus_edu_sg/IQBk055rXrOJQKKp24CwlueuAW7GmwTP7Zk2jsJF-KNrduA?e=FRHB8k

The separately submitted 5-7 minute demo video remains compulsory.

- **Booking:** Any one team member may reserve a slot on behalf of the whole team by entering only the team number in one empty slot. Use one slot per team and do not overwrite another team's entry. If you move your booking, clear your old slot first.
- **Attendance:** All team members must attend the same booked slot and should be ready at least 5 minutes before the scheduled time.
- **Individual assessment:** Questions may be directed to any member and the viva component is individually assessed; members of the same team may receive different viva marks.
- Technical readiness: Have the server, browser client, deterministic caption_source.py demo, source code, logs/results, and Ubuntu test environment ready before the slot begins. Setup time is part of the 15-minute slot.

Example technical questions include:

- Why can an older revision never overwrite a newer one?
- What exactly happens when a client reconnects after 10 seconds offline?
- Where is per-client buffering implemented, and how is it bounded?
- Why did P95/P99 increase under packet loss even though WebSocket/TCP is reliable?
- What happens when a client consumes messages more slowly than they are generated?
- Which provisional revisions can safely be dropped, and why?
- How would your behavior change if bandwidth were cut in half?
- Show the code implementing your overload/backpressure policy.
- Explain one experimental result that surprised your team.

The instructor may request a small, bounded modification or ask a member to identify the code location that would need to change. This is intended to assess understanding, not typing speed.

16. Suggested Three-Week Plan

Week	Target
Week 1	End-to-end caption_source.py -> server -> browser; protocol; revision replacement; basic UI; automatic reconnect; first working deterministic demo.
Week 2	Rolling history and What did I miss?; reconnect/resume and deduplication; bounded per-client queues; slow-client/backpressure policy; 100-client public tests.
Week 3	Optimization; tc/netem experiments; repeated measurements and before/after comparison; report/demo video; submission compatibility check; viva preparation.

Design principle: The aim is not to maximize the number of features. The aim is to build a usable networked application that remains correct, fresh, and understandable when conditions become difficult.

17. Optional Bonus Feature - Real-Lecture ASR Integration

This section is optional and is not required for the 100 mandatory marks. Teams may implement a real-lecture Automatic Speech Recognition (ASR) input path for up to 3 bonus marks. The overall Assignment 4 mark remains capped at 100.

- Use a local/open-source ASR engine or library (for example, faster-whisper, whisper.cpp, Vosk, or another suitable tool) to generate captions while a real lecture video is playing. Pre-generated subtitle playback alone does not qualify.
- Feed the generated captions through the same LiveRecall protocol, solution_server.py, and browser client used for the mandatory deterministic assignment; do not build a separate mock-up.
- Demonstrate at least a representative 3-minute continuous excerpt. ASR accuracy/model size is not graded; the bonus rewards correct integration and a usable end-to-end pipeline.
- Keep bonus-feature dependencies separate so that the mandatory requirements.txt / solution_server.py evaluation path remains lightweight and reproducible.
- Suggested bonus allocation: 1 mark for correct same-system integration, 1 mark for a working continuous lecture-video demonstration, and 1 mark for clear explanation of timing/integration limitations. Bonus marks do not compensate for weak hidden networking performance or viva understanding.